Speaker Script

Suleyman's Sections

CAS 760 Development Project 2

A Simple File System in Alonzo

Target timing: ∼7 minutes total, ∼60 seconds per slide.

My slides, in order (deck slides 8–14):

1. Theory for Files — FILE
2. Theory Extension for Secured Files — SECURED-FILE
3. Theory Extension for Directories — DIRECTORY
4. Directories as an Inductive Type
5. Directory Development — DIR-DEV (both `contains` and `size` defined here)
6. Explanation for Directory Development
7. Example of Directory Evaluation (handoff to Hamza)

How to read this:

- The *italic indented text* is what I say **verbatim**.
- → **Red arrows** are stage directions — where to point, gesture, or pause.
- *(pause)* means take a breath, about one second.
- ⇒ **Green TRANSITION** means advance to the next slide.
- Each slide starts on a fresh page — flip as you go.

Read this aloud at least twice before the talk. Time yourself.

SLIDE 1 · Files — Theory FILE

timing: ∼70 sec

ON SCREEN: *Base types* N, C, F. *Constants* mkfile, fname, fcontent *with types. Seven axioms (1–7) listed.*

"Thanks Hamza. Now I'll walk through the Files theory."

→ **Brief nod toward Hamza. Face the audience.**

"The idea here is straightforward: a file is a named container for content. Conceptually, a file is just a pair — a name and some content — but we want to formalize that abstractly, without committing to what names or contents actually are."

"So we introduce three base types —"

→ **Point to the line "N, C, F" on the slide.**

"— N for names, C for content, and F for the file type itself. We keep them opaque on purpose, so that this theory captures the ***structure*** *of files rather than any specific representation."*

(pause)

"We have one constructor, mkfile *—"*

→ **Point to the** mkfile **line.**

"— of type $N \to C \to F$, which builds a file from a name and content. And we have two accessors, fname *and* fcontent *—"*

→ **Point to the** fname **and** fcontent **lines in turn.**

"— that pull the name and content back out."

"The axioms say what you'd expect. Axioms 1 and 2: mkfile *is total and injective — every name-content pair gives a valid file, and distinct pairs give distinct files. Axioms 3 and 4 are the two accessor axioms:*

if you build a file with mkfile *n c, extracting the name gives you n back, and the content gives you c. Axioms 5 and 6 say the accessors are themselves total. And Axiom 7 — surjectivity — says every file f is equal to* mkfile *applied to its own name and content. That closes the loop: F is isomorphic to $N \times C$, but presented abstractly as a theory."*

⇒ **TRANSITION:** advance to Secured Files slide.

SLIDE 2 · Secured Files — Theory Extension SECURED-FILE

timing: ∼75 sec

ON SCREEN: *"Extends: FILE and PERM". "New base types: none (P inherited from PERM)". New constants* perm, protect. *Axioms 8–12.*

> *"SECURED-FILE combines files and permissions. This is one of the more interesting design points in the paper."*

> *"The intuition is: FILE plus PERM equals SECURED-FILE. Each file gets a permission level attached to it. Formally, we present SECURED-FILE as a* ***joint extension*** *of both FILE and PERM —"*

→ **Point to the "Extends: FILE and PERM" line on the slide.**

> *"— this is a pushout-style construction in the little theories method."*

> *"What that means concretely: SECURED-FILE has* ***no new base types*** *of its own."*

→ **Point to "New base types: none" line.**

> *"It inherits P from PERM and inherits N, C, F from FILE. The two extension arrows in the theory graph — from PERM to SECURED-FILE, and from FILE to SECURED-FILE — encode that inheritance formally."*

(pause)

> *"What SECURED-FILE* ***does*** *add is two new constants."*

→ **Point to** perm **line.**

> *"First,* perm*, of type $F \to P$, which tells you the permission of a file."*

→ **Point to** protect **line.**

"And second, protect*, of type $F \to P \to F$, which takes a file and a permission and gives you back the same file with that permission assigned."*

"Axioms 8 through 12 govern these. Eight and nine are totality. Ten is the core semantics: applying protect *with permission p gives a file whose permission is p. And eleven and twelve say that* protect *only changes the permission — it preserves the name and the content. So* protect *is a pure permission-update operation."*

⇒ **TRANSITION:** advance to Directories slide.

SLIDE 3 · Directories — Theory Extension DIRECTORY

timing: ∼70 sec

ON SCREEN: *"Design choice: flat structure." Base type D. Constructors* emptydir, addfile. *Axioms 13–16.*

> *"DIRECTORY is a theory extension over SECURED-FILE. This is where the design choice I want to highlight comes in: we model directories as a* ***flat*** *structure, not as a hierarchical tree."*

> *"The reason is that this keeps the inductive type clean."*

→ **Point to the** addfile **type signature on the slide.**

> *"The constructor* addfile *has type $F \to D \to D$ — it takes a* ***file****, not a directory, as its first argument. That gives us a direct analogy with the list type from the textbook:* emptydir *plays the role of* nil*, and* addfile *plays the role of* cons*."*

> *"If we wanted a hierarchical model, we'd need a second constructor,* adddir *of type $D \to D \to D$ —"*

→ **If whiteboard available: quickly write "`adddir : D → D → D`" in the corner. Otherwise just emphasize with hand gesture.**

> *"— which would make this a tree-shaped inductive type. That's a natural extension, and we mention it as future work in the paper, but for this project the flat model is the right scope."*

(pause)

> *"The axioms are standard. Axiom 13 says* addfile *is total. Axiom 14 says it's* ***injective*** *— and we'll see later that this injectivity is what makes the FILE-to-DIR morphism work."*

→ **Slight emphasis on "injective" — this callback matters for Hamza's morphism section.**

"Axiom 15 is disjointness: addfile *f d is never equal to* emptydir*. And Axiom 16 is the structural induction principle, which lets us prove properties over all directories by handling the two constructor cases."*

⇒ **TRANSITION:** advance to Inductive Type verification slide.

SLIDE 4 · Directories as an Inductive Type

timing: ∼70 sec

ON SCREEN: $K_D = \{\text{emptydir}, (\text{addfile}, U_F)\}$. *Interpretation bullets. Conclusion about finite lists of* D_F^M.

> *"To confirm DIRECTORY is a proper recursive inductive type in Alonzo, we follow the procedure from Chapter 10 of Farmer's textbook."*

> *"The constructor set is* K_D *—"*

→ **Point to the $K_D = \{\ldots\}$ expression on the slide.**

> *"— which contains* emptydir *and the pair* $(\text{addfile}, U_F)$. emptydir *is a* ***first-kind*** *constructor — it's nullary. And* $(\text{addfile}, U_F)$ *is a* ***fourth-kind*** *constructor: the* U_F *means the first argument ranges over the entire domain of* F*, and because* D *appears in the domain of* addfile*'s type, it makes* addfile *recursive."*

(pause)

> *"We check the seven obligations from Chapter 10 in the paper. I'll just mention them briefly: the quasitype for the* F*-argument is non-empty because* F *is a base type; totality and injectivity hold by Axioms 13 and 14;* emptydir *is not in the range of* addfile *by disjointness; disjoint-ranges is trivial since there's only one non-nullary constructor; and structural induction is Axiom 16. All seven pass."*

> *"The consequence, by the Inductive Types Theorem, is that in every standard model* D *is isomorphic to the set of* ***finite lists*** *over the interpretation of* F*. So directories really do behave like lists of files — exactly as the constructor signatures suggested."*

⇒ **TRANSITION:** advance to DIR-DEV intro slide.

SLIDE 5 · Directory Development — DIR-DEV

timing: ∼90 sec

ON SCREEN: *Name: DIR-DEV. Bottom theory: DIR. Def 1 (*contains*) with Thms 1–2. Def 2 (*size*) with Thms 3–6. All on one slide.*

> *"DIR-DEV is a* ***development*** *over DIRECTORY — not another theory extension. That matters because* `contains` *and* `size` *are* ***derived*** *operations, definable from* addfile *and* emptydir*. So we* ***prove*** *they exist rather than postulating them as new theory constants. In Project 1 we lost points for making the opposite mistake — this is the direct correction."*

→ **Brief pause on the "lesson learned" beat — earns goodwill.**

> *"The slide gives both definitions and all six theorems. I'll walk through them."*

→ **Point to Def 1 (`contains`) on the slide.**

> *"Def 1:* `contains` *of type* $N \to D \to \mathbb{B}$*. Defined as the unique function* g *satisfying two recursion clauses —* $g\ n$ emptydir *is false, and* $g\ n$ (addfile $f\ d$) *is true if* fname f *equals* n*, or if* $g\ n\ d$ *is already true. So* `contains` *either matches the outermost file's name, or recurses into the tail."*

→ **Move pointer to Thms 1–2.**

> *"Theorem 1 says* `contains` *is total. Theorem 2 says the empty directory contains no name."*

(pause)

→ **Point to Def 2 (`size`) on the slide.**

> *"Def 2:* `size` *of type* $D \to \mathbb{N}$*. Same iota-term pattern —* g emptydir *equals zero, and* g (addfile $f\ d$) *equals* $g\ d$ *plus one."*

→ **Move pointer down the four theorems 3–6.**

"Four theorems: Theorem 3, totality. Theorem 4, size of empty is zero. Theorem 5, the recurrence — `size` (addfile f d) *equals* `size` d *plus one. And Theorem 6, non-negativity, which follows from the recurrence by structural induction. Both definitions are validated in the appendix."*

⇒ **TRANSITION:** advance to the Explanation slide.

SLIDE 6 · Explanation for Directory Development

timing: ∼50 sec

ON SCREEN: *Plain-language prose summary of* `contains` *and* `size`*. Two bullet blocks. Mentions appendix validation.*

> *"This slide restates the same two definitions in plain language for intuition."*

→ **Point to the `Contains` block on the slide.**

> *"*`contains` *n d asks: is there a file named n somewhere in directory d? The empty directory contains nothing; and* addfile *f d contains n exactly when either f's own name is n, or d already contained n."*

→ **Point to the `Size` block.**

> *"*`size` *d counts file entries: zero for* emptydir*, and one more than* `size` *d whenever we prepend a file with* addfile*."*

(pause)

> *"Both definitions are iota-terms — definite descriptions. The appendix discharges existence and uniqueness for each, which is what makes them legal development-level abbreviations."*

⇒ **TRANSITION:** advance to the concrete example slide.

SLIDE 7 · Example of Directory Evaluation

timing: ∼70 sec

ON SCREEN: *Two files* f_1, f_2 *built via* protect(mkfile n c) p. *Two-entry directory* d. *Values of* `contains` *at* n_1, n_2, n_3 *and* `size` $d = 2$.

"To close out, a concrete evaluation."

→ **Point to f_1 and f_2 definitions on the slide.**

"Let f_1 *be* protect *applied to* mkfile n_1 c_1 *with* ***read*** *permission. Let* f_2 *be* protect *applied to* mkfile n_2 c_2 *with* ***write*** *permission. Let* d *be* addfile f_1 (addfile f_2 emptydir) *— a two-entry directory."*

→ **Point to each `contains` equation.**

"`contains` n_1 d *is* ***true*** *—* f_1*'s name matches at the outer layer.* `contains` n_2 d *is also* ***true*** *— we recurse into the tail and find* f_2*.* `contains` n_3 d *is* ***false*** *— no match at either layer."*

→ **Point to the `size` line.**

"And `size` d *equals* ***two*** *— each* addfile *contributes one,* emptydir *contributes zero. So the two-entry directory has size two, confirming Theorem 5."*

"That completes my sections of the development. Hamza will now take over with the theory morphism."

⇒ **HAND OFF:** look to Hamza, step slightly aside, smile.

Q&A PREP · Likely Questions

each answer has a **plain** short form and a **technical** follow-up — lead with plain, switch to technical if they push for depth

★ TOP PRIORITY — ALMOST CERTAIN FARMER ASKS THIS

Farmer asked the previous group: "Using Alonzo and the little-theories machinery, what **helped** and what **hurt**?" — rehearse this first.

My short answer — what helped:

> *"Honestly, the biggest win was the **little-theories method** itself. Instead of one giant theory doing everything, we broke the system into four small ones — FILE, PERM, SECURED-FILE, DIRECTORY — plus a development on top. Each piece was small enough to actually reason about on its own, and the theory-graph arrows made the dependencies obvious. The other thing that really helped was **Chapter 10's recipe** for inductive types — it gave us a seven-item checklist to verify DIRECTORY is a proper recursive type, instead of us having to invent the proof strategy from scratch. And once Hamza's theory morphism is set up, the **Morphism Theorem** transports theorems between theories automatically — so we don't have to re-prove anything we already proved in FILE."*

My short answer — what hurt:

> *"The hard parts were mostly about what Alonzo **doesn't** have. There's no pattern matching, no recursive-function keyword like you'd find in OCaml or Haskell — so to define* `contains` *and* `size`*, I had to write iota-terms and then **separately** prove existence and uniqueness in the appendix. That basically doubled the work for those definitions. The theory morphism was also trickier than I expected —* extract *is only defined on single-entry directories, so we had to introduce a closed quasitype ζ_F to pin the domain down. And things that are obvious in a programming language — like* `size` $d \geq 0$ *— still required structural induction, because Alonzo's $\mathbb{N}$ doesn't bake in non-negativity."*

Deeper — what helped, technically:

> *"Four things at a technical level. First, **conservative extension** —*

theorems proved in FILE lift for free into SECURED-FILE and DIRECTORY along the extension arrows, so we never re-prove accessor or surjectivity properties. Second, the ***Inductive Types Theorem*** *from Chapter 10 — we discharged seven syntactic obligations and got* $D \cong \mathrm{list}(F)$ *in every standard model as a consequence, without needing a custom semantic argument. Third,* ***iota-terms*** *as the native way to introduce derived operations — prove existence-and-uniqueness once, and the defined symbol behaves like any other Alonzo constant. And fourth,* ***quasitypes****, which let us restrict the theory morphism's source domain cleanly to exactly* ζ_F *without breaking anything."*

Deeper — what hurt, technically:

"Three main friction points. First, no fixpoint operator — every recursive definition becomes an iota-term plus a validation proof, which roughly doubled the appendix. Second, the theory-translation machinery is strict about totality — we had an earlier version of the morphism that mapped $F \mapsto D$*, and only during review we realized that makes* $\mathrm{fname} \circ \mathrm{extract}$ *undefined on multi-entry directories. Fixing it required identifying the exact closed quasitype* ζ_F *where* extract *is total. And third,* ***scoping in the theory graph*** *— choosing DIRECTORY's bottom theory as SECURED-FILE rather than FILE directly matters for what symbols are available inside* addfile*'s type, and that kind of decision has non-obvious ripple effects downstream on the inductive-type obligations."*

Q1. Why flat directories and not hierarchical?

Plain:

"The project is called a ***simple*** *file system. Flat means a directory is just a list of files — like a shopping list. Hierarchical would mean folders inside folders, which is out of scope for this project."*

Technical:

"Flat keeps the inductive type within the standard fourth-kind constructor pattern of Chapter 10. It preserves the direct list analogy — emptydir *is* nil, addfile *is* cons *— and it makes the FILE-to-DIR*

morphism clean because single-entry directories give a canonical embedding target. Hierarchy would require a second recursive constructor adddir : $D \to D \to D$*, which breaks the list analogy and makes* extract *ambiguous on non-leaf branches. We note hierarchy as future work."*

Q2. Why are `contains` and `size` in a development instead of extending DIRECTORY?

Plain:

> *"Because we can* ***define*** *them from the primitives we already have. If we added them as new theory constants, we'd be postulating things we can already prove — that's redundant, and axiomatically wasteful."*

Technical:

> *"*`contains` *and* `size` *are* ***derived*** *operations, definable from* addfile *and* emptydir *via iota-terms. Putting them in DIRECTORY as primitives would enlarge the theory's signature and require new axioms for properties we already prove from Axioms 13–16. The development framework lets us define via ι, discharge existence-and-uniqueness once, and use the symbol freely — without expanding the theory. In Project 1 we made the opposite mistake and lost points; this is the direct correction."*

Q3. How did you verify the DIRECTORY obligations?

Plain:

> *"Chapter 10 of Farmer's book gives a checklist for what makes something a valid inductive type. We went through each item and showed our axioms satisfy it."*

Technical:

> *"All seven obligations: non-empty quasitypes for each constructor argument, totality of each constructor, injectivity of each non-nullary constructor, disjointness of ranges between constructors, and structural induction. Each discharges from our four axioms — Axiom 13 covers totality of* addfile*, Axiom 14 injectivity, Axiom 15 the disjointness of* addfile*'s range from* emptydir*, and Axiom 16 structural induc-*

tion. The non-emptiness obligation for the F-argument holds because F is a base type and every base type is inhabited in any Alonzo frame. The disjoint-ranges obligation between multiple non-nullary constructors is trivial since we only have one. Full check is in the appendix."

Q4. What if two files in a directory have the same name?

Plain:

> *"Nothing prevents it. The theory allows duplicates.* `contains` *just returns true as soon as it finds any match."*

Technical:

> *"The inductive type D has no uniqueness invariant — two* addfile *applications can wrap files with identical names because* addfile *is only required to be injective on its pair of arguments, not on file names.* `contains` *n d is an existential predicate — it reports* at least one *match. Adding name-uniqueness would be a development-layer invariant — a provable predicate over d — not a theory-layer axiom. That's reasonable future work."*

Q5. Why use K_D for the constructor set?

Plain:

> *"It's just notation. K stands for Konstruktor — the German word for constructor — and the subscript says which type's constructors we're talking about."*

Technical:

> *"Standard Chapter 10 convention. K for Konstruktor, subscripted by the inductive type to disambiguate when multiple types are in play. Subscript D avoids clashing with the base type C of FILE, and lets us distinguish K_D from, say, the implicit constructor set of F which is just* {mkfile}*."*

Q6. What does "pushout-style construction" mean?

Plain:

"It's a way to combine two theories that don't overlap, then add new things on top. FILE and PERM don't share anything, so we just glue them together and add the two new constants perm *and* protect*."*

Technical:

"Category-theoretic term for the colimit of two objects over a shared sub-object. Here the shared sub-theory is essentially empty — FILE and PERM have disjoint signatures — so the pushout is just their disjoint union extended with the new SECURED-FILE constants and axioms. In the little-theories graph, we encode this by having SECURED-FILE extend both *parents simultaneously — the two arrows from FILE and PERM into SECURED-FILE in our theory graph are exactly those two extension morphisms. Semantically: the language of SECURED-FILE is* $\Sigma_{\mathrm{FILE}} \cup \Sigma_{\mathrm{PERM}} \cup \{\mathrm{perm}, \mathrm{protect}\}$*."*

Q7. What is Alonzo?

Plain:

"It's Dr. Farmer's name for a specific version of Church's simple type theory — a logic for reasoning about typed functions. It's the system the whole course is built on."

Technical:

"Alonzo is a Church-style simple type theory defined in Farmer's textbook. Key features: a hierarchy of types built over base types and function arrows; the iota-operator for definite descriptions; quasitypes for expressing subtype-like constraints without full dependent types; a well-defined treatment of undefinedness via the frame definition; and a proof-theoretic system that makes the ***little theories method*** *tractable — you build a graph of small theories connected by extensions and translations, and transport theorems along the arrows."*

Q8. What's the difference between a theory extension and a development?

Plain:

"An extension adds new ***primitives*** *— new base types, constants, and axioms. A development only adds* ***definitions and theorems*** *—*

things you can already prove from the theory below it. A development can never introduce a new axiom."

Technical:

"An extension $T' \supseteq T$ enlarges the signature (new base types and/or constants) and the axiom set — strictly more primitive symbols and assumptions. A development Δ over a bottom theory *T is a sequence of definitions (iota-term abbreviations with discharged existence-and-uniqueness) and theorems provable from T's axioms alone. Crucially: everything in Δ is conservative over T — no new axioms, no new primitive symbols. That's the reason* `contains` *and* `size` *belong in DIR-DEV, not in a DIRECTORY-prime extension."*

Q9. Why does injectivity of addfile matter?

Plain:

"It means different inputs give different outputs — no two calls to addfile *with different arguments can produce the same directory. That's what lets Hamza's theory morphism pull the unique file back out of a single-entry directory."*

Technical:

"Axiom 14 is injectivity of addfile *on its pair of arguments (f, d). In Hamza's morphism section,* extract *is defined as an iota-term — the unique $f : F$ such that $d =$* addfile f emptydir*. Without injectivity, multiple f-values could satisfy that equation, and the iota-term would be undefined. Injectivity is exactly what makes* extract *total on the closed quasitype ζ_F of single-entry directories, which is the domain that makes the FILE-to-DIR translation well-formed."*

Q10. What is a definite description (iota-term)?

Plain:

"It's the expression "the unique x such that $P(x)$." You say what property you want, and if exactly one thing satisfies it, the iota-term denotes that thing. If zero or multiple things satisfy it, the term is undefined."

Technical:

> *"In Alonzo, $\iota x : A.\ P(x)$ denotes the unique element of type A satisfying predicate P when such an element exists and is unique; otherwise the term is undefined. We use iota-terms to define* `contains` *and* `size` *via recursion schemes: the definition says* "the unique function g satisfying these two recursion clauses." *The validation step proves existence and uniqueness, which licenses the iota-term as a legal abbreviation. In Hamza's section,* extract $d = \iota f : F.\ d =$ addfile f emptydir, *which is total exactly on ζ_F."*

Q11. How does structural induction work here (Axiom 16)?

Plain:

> *"To prove something is true for* ***every*** *directory, we only need to check two cases: it's true for the empty directory, and if it's true for d, then it's true for* addfile f d *too. Axiom 16 says that's enough."*

Technical:

> *"Axiom 16 is the induction principle for D: for any predicate $p : D \to \mathbb{B}$, if p* emptydir *holds and $p\ d \Rightarrow p$* (addfile f d) *for all $f : F, d : D$, then $p\ d$ holds for every $d : D$. It's what lets us prove Theorem 6 (size non-negativity) from Theorem 4 (base case) and Theorem 5 (the recurrence). It's also the 7th inductive-type obligation — without it, D wouldn't be recognized as a proper recursive inductive type by the Inductive Types Theorem."*

Q12. Why model directories as an inductive type (instead of just reusing the textbook's list type)?

Plain:

> *"Because a directory isn't* any *list — it's a list of* files *specifically, which are built on top of our SECURED-FILE theory. Inductive-typing D directly in our theory graph keeps everything self-contained and lets us prove directory-specific theorems without importing the generic list theory."*

Technical:

"Two reasons. First, pedagogical: the assignment asks us to define a recursive data type in Alonzo*, and going through Chapter 10's constructor-set formalism — K_D, the seven obligations, the Inductive Types Theorem — is exactly the exercise. Second, theoretical: by axiomatizing D directly in DIRECTORY rather than deriving it as* $\mathrm{list}(F)$*, we get* emptydir *and* addfile *as primitive constants of our theory, which makes Hamza's FILE-to-DIR morphism a translation within our own graph, not a cross-reference to an external list theory. The Inductive Types Theorem still gives us $D \cong \mathrm{list}(F)$ in every standard model — we get the list isomorphism as a* consequence*, not an assumption."*

Q13. Why have SECURED-FILE as a separate theory instead of baking permissions into FILE?

Plain:

"Separation of concerns. FILE is about the structure of a file — name plus content. PERM is about permission levels. If we baked permissions into FILE, we'd lose the ability to reason about unsecured files, and we'd violate the little-theories principle of keeping each theory as small as possible."

Technical:

"The little-theories method says: each theory should axiomatize exactly one concept. FILE axiomatizes the abstract file record; PERM axiomatizes permission levels; SECURED-FILE is their joint extension *that adds the connection between them. This gives us three benefits. One: FILE is reusable in contexts that don't care about permissions. Two: the theory graph encodes the dependency structure explicitly — anyone reading it sees that permissions are an added concern, not fundamental to files. Three: theorems proved in FILE transport* automatically *into SECURED-FILE via the extension arrow, so we don't re-prove them."*

Q14. Why does DIRECTORY extend SECURED-FILE and not FILE directly?

Plain:

"Because in our file system, every *file in a directory has a permission attached. If DIRECTORY extended FILE instead, we'd have files without permissions appearing in directories, which doesn't model what we want."*

Technical:

"DIRECTORY's addfile *has signature $F \to D \to D$, and F lives in DIRECTORY's language via the extension arrow from its bottom theory. Since DIRECTORY extends SECURED-FILE, the F in scope is the permission-enriched FILE type — every file that* addfile *can wrap has a permission accessible via* perm*. If DIRECTORY extended FILE, the F would be plain FILE without* perm *or* protect*, and we couldn't write security-sensitive operations at the directory level. The theory graph exactly models the real-world constraint that file-system directories contain secured files."*

Q15. Why are the base types N, C, F, P abstract instead of concrete (strings, bytes)?

Plain:

"Because we want the theory to describe any *file system, not just one with a specific choice of name format or content type. Abstraction lets us capture the structural properties without being tied to a representation."*

Technical:

"This is the standard little-theories approach: axiomatize the minimum. By leaving N, C, F, P as base types, our axioms express the structural *relationships — totality, injectivity, accessor-constructor coherence, surjectivity — that any concrete instantiation must satisfy. A user who wants strings-as-names and bytes-as-content builds a theory translation from FILE into that concrete setting; our theorems transport along it via the Morphism Theorem. Making the types concrete would bind the theory to one representation and break reusability."*

Q16. Why does FILE have a surjectivity axiom (Axiom 7)?

Plain:

> *"Because without it, there could be files in F that weren't* built *by* mkfile *— phantom files floating around. Surjectivity says every file f equals* mkfile *applied to its own name and content, so nothing extra exists."*

Technical:

> *"Axiom 7 —* $\forall f : F.\ f = \text{mkfile}\ (\text{fname}\ f)\ (\text{fcontent}\ f)$ *— closes the loop between the accessors and the constructor. Without it, the accessor axioms (3 and 4) only constrain F on the image of* mkfile*; surjectivity says the image is* all *of F. Combined with totality (1) and injectivity (2), it gives us $F \cong N \times C$ as a bijection in every standard model, not just an injection. That's what lets us treat F as a record type rather than just an opaque set of things with name-and-content accessors."*

Q17. Why did you use iota-terms (definite descriptions) for `contains` and `size` instead of explicit recursion?

Plain:

> *"Alonzo doesn't have built-in pattern-matching or recursive-function syntax like a programming language. Iota-terms are the standard way to name the unique function satisfying a given recursion — and the Inductive Types Theorem guarantees that unique function exists."*

Technical:

> *"Alonzo is a pure simple type theory — there's no native* fix*-operator or pattern-matching syntax. Recursive definitions are encoded as iota-terms asserting the existence of a unique function satisfying the recursion equations. The Inductive Types Theorem guarantees this function exists and is unique when the recursion is primitive-recursive over an inductive type — which is exactly our case. The validation step in the appendix is discharging the existence-and-uniqueness obligation for each iota-term."*